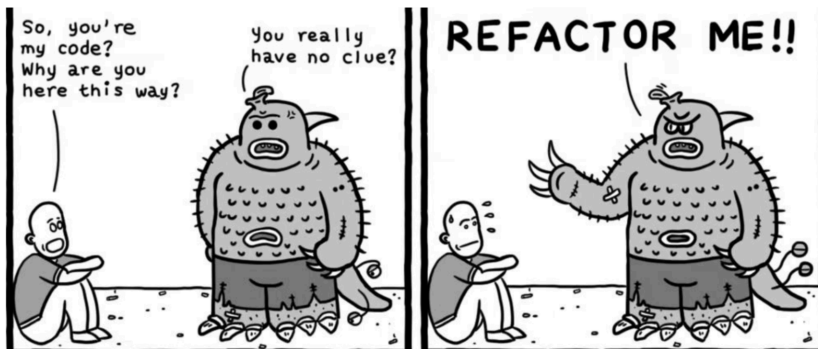


REFACTORING

“ La refactorización (refactoring) es el proceso de reorganizar el código existente para mejorar su estructura interna sin alterar su comportamiento externo. Su propósito es hacer el software más comprensible, mantenible y económico de modificar, reduciendo la deuda técnica y evolucionando el sistema hacia un diseño más limpio y simple. ”



Refactoring.com



REFACTORING

1

CLASE GRANDE

2

CÓDIGO DUPLICADO

3

MÉTODO LARGO (MAIN)

4

OBSESIÓN PRIMITIVA

5

**NIVELES DE ABSTRACCIÓN
HETEROGÉNEOS**

6

ENCAPSULAMIENTO DEFICIENTE

1) CLASE GRANDE

Separación de responsabilidades: el usuario conserva sus datos y la cartera maneja el dinero.

Antes

Clase monolítica

```
class User:
    def __init__(self, name, email, address, credit_card, pin, billing_info):
        self.name = name
        self.email = email
        self.address = address
        self.credit_card = credit_card
        self.pin = pin
        self.billing_info = billing_info
        self.wallet_balance = 0
        self.rentals = []
        self.purchases = []
        self.notifications = []
```

Después

Responsabilidad separada

```
class Wallet:
    def __init__(self):
        self.balance = 0

    def credit(self, amount):
        self.balance += amount

    def debit(self, amount):
        if self.balance < amount:
            raise ValueError("Insufficient balance")
        self.balance -= amount

class User:
    def __init__(self, name, email):
        self.name = name
        self.email = email
        self.wallet = Wallet()
```

Referencia: [extractClass](#)

2) CÓDIGO DUPLICADO

Centralización del pago para evitar repetir la misma validación en cada operación.

Antes

Lógica repetida

```
def register_rental(self, movie):
    if self.wallet_balance >= movie.rental_price:
        self.wallet_balance -= movie.rental_price
        ...
    else:
        print("Insufficient balance")

def register_purchase(self, movie):
    if self.wallet_balance >= movie.purchase_price:
        self.wallet_balance -= movie.purchase_price
        ...
    else:
        print("Insufficient balance")
```

Después

Un solo punto de pago

```
class PaymentProcessor:
    def process(self, wallet, amount):
        wallet.debit(amount)

def register_rental(self, movie, payment_processor):
    payment_processor.process(self.wallet, movie.rental_

def register_purchase(self, movie, payment_processor):
    payment_processor.process(self.wallet, movie.purchas
```

Referencia: [replaceInlineCodeWithFunctionCall](#) (*)

3) MÉTODO LARGO (MAIN)

Separación por casos de uso: preparar usuario y procesar transacciones en funciones distintas.

Antes

Flujo concentrado

```
def main():
    user = User(...)
    movie1 = Movie("Inception", 3.99, 14.99)
    movie2 = Movie("The Matrix", 2.99, 12.99)

    user.add_money_to_wallet(20)
    user.register_rental(movie1)
    user.register_purchase(movie2)

    user.get_rentals()
    user.get_purchases()
```

Después

Funciones por intención

```
def setup_user():
    user = User("John Doe", "john@example.com")
    user.wallet.credit(20)
    return user

def process_transactions(user):
    movie1 = Movie("Inception", 3.99, 14.99)
    movie2 = Movie("The Matrix", 2.99, 12.99)

    payment = PaymentProcessor()

    user.register_rental(movie1, payment)
    user.register_purchase(movie2, payment)

def main():
    user = setup_user()
    process_transactions(user)
```

Referencia: [splitPhase](#)

4) OBSESIÓN PRIMITIVA

Modelado explícito para reemplazar estructuras anónimas con un objeto de dominio claro.

Antes

Diccionario ad hoc

```
rental = {  
    'movie': movie,  
    'start_date': datetime.datetime.now(),  
    'end_date': datetime.datetime.now() + datetime.timedelta(days=1),  
    'views_remaining': 1  
}
```

Después

Objeto de dominio

```
class Rental:  
    def __init__(self, movie):  
        self.movie = movie  
        self.start_date = datetime.datetime.now()  
        self.end_date = self.start_date + datetime.timedelta(days=1)  
        self.views_remaining = 1  
  
rental = Rental(movie)  
self.rentals.append(rental)
```

Referencia: [replacePrimitiveWithObject](#)

5) NIVELES DE ABSTRACCIÓN HETEROGÉNEOS

Separación en servicio de dominio para no mezclar reglas de negocio con detalles técnicos.

Antes	Mezcla de responsabilidades	Después	Servicio de dominio
	<pre>def register_rental(self, movie): if self.wallet_balance >= movie.rental_price: self.wallet_balance -= movie.rental_price rental = {...} self.rentals.append(rental)</pre>		<pre>class RentalService: def rent(self, user, movie, payment_processor): payment_processor.process(user.wallet, movie.rental_price) rental = Rental(movie) return rental rental = rental_service.rent(user, movie, payment) user.rentals.append(rental)</pre>

Referencia: [extractFunction](#)

6) ENCAPSULAMIENTO DEFICIENTE

La lógica de pago debe vivir en el comportamiento del objeto, no en campos expuestos directamente.

Antes

Acceso directo al estado

```
class PaymentProcessor:
    def process_payment(self, user, amount):
        if user.wallet_balance >= amount:
            user.wallet_balance -= amount
            return True
```

Después

Encapsulación por comportamiento

```
class User:
    def pay(self, amount):
        self.wallet.debit(amount)

class PaymentProcessor:
    def process(self, user, amount):
        user.pay(amount)
```

Referencia: [encapsulateVariable](#), [hideDelegate](#)